bank.ly Security Self-Assessment

2026 Shenzhen International FinTech Competition (FinTechathon)

Artificial Intelligence Track

Team SKS

GitHub repository: github.com/fvia88/ai-banking-agent

1 Assessment scope and security architecture

bank.ly demonstrates conversational balance queries, spending analysis, transfers and subscription management using synthetic banking data. This assessment describes permission-tier enforcement, execution safeguards, model boundaries, audit coverage and the conditions of the shared judging deployment. It is a project self-assessment, not an independent security certification.

The language model proposes structured operations; it cannot directly move funds or modify subscriptions. A deterministic policy engine evaluates proposals against backend-resolved context. The backend owns approval records, verification challenges, database transactions and execution results.

Security responsibilities

Engineering role	Responsibility
Agent Engineering	Validated intent extraction, verified-context matching, correlated tool results, safe failure replies and controlled task continuation.
Backend Engineering	Identity scoping, approval binding, execution-time checks, transaction protection, persistence and audit events.
Security Engineering	Fail-closed policy evaluation, permission-tier rules, lock policy and adversarial policy checks.
Finance Engineering	Deterministic financial calculations and verification-challenge implementation in coordination with the backend.
Subscription Engineering	Recurring-charge detection, stored-record mapping and cancellation eligibility.

Authorization boundary

`evaluate_action(proposal, context, config)` is a pure policy function. It performs no I/O, does not mutate its inputs and returns a rejection decision for invalid input or unexpected policy errors. The backend uses the real security adapter by default; mock security requires explicit configuration. The shared hosted entry point requires real agent, security and subscription adapters.

Sections 2-3 describe authorization and execution. Sections 4-5 cover model and deployment protections, verification evidence and operational boundaries.

Implementation references: `security/policy.py`, `security/rules.py`, `backend/app/services/adapters.py`, `infra/start_shared.py`.

2 Permission tiers and input validation

Permission-tier implementation

Tier	Applicable operations	Required control
GREEN	Balance, spending and recurring-payment queries.	Validate tool arguments and account scope before automatic read execution.
YELLOW	Subscription cancellation; transfers whose completed daily total plus requested amount is at most CNY 1,000.00.	Explicit confirmation of the exact stored proposal.
RED	Transfers whose cumulative daily total exceeds CNY 1,000.00.	Explicit confirmation followed by simulated MFA.
Rejection	Invalid input, unknown tool, invalid arguments, account mismatch, prohibited locked-account operations or internal policy error.	`allowed_to_proceed=False`; no financial execution.

A RED tier alone is not permission to execute. The backend must inspect `allowed_to_proceed` and enforce the confirmation and verification requirements. Subscription cancellation remains a confirmation-required operation.

Fail-closed evaluation sequence

Step	Check	Failure or decision outcome
1	Require `ActionProposal` and `BankingContext` objects.	Reject with `INVALID_PROPOSAL` or `INVALID_CONTEXT`.
2	Check tool membership in `contracts.enums.ToolName`.	Reject with `UNKNOWN_TOOL`.
3	Run `validate_tool_arguments` against the canonical schema.	Reject with `INVALID_TOOL_ARGUMENTS`.
4	Check `account_locked` for mutation tools; optionally for reads under policy configuration.	Reject with `ACCOUNT_LOCKED`.
5	Compare `source_account_id` or queried `account_id` with context.	Reject with `SOURCE_ACCOUNT_MISMATCH` or `ACCOUNT_MISMATCH`.
6	Use Decimal to evaluate completed daily total plus requested amount.	Return the applicable permission tier and required controls.
Guard	Catch unexpected policy exceptions.	Reject with `INTERNAL_POLICY_ERROR`.

Unexpected evaluation errors return a rejection rather than permissive fallback. Read access remains available under the default account-lock policy.

Canonical argument controls

Input	Validation
Transfer amount	Positive Decimal, at most two decimal places and 16 total digits; reject zero, negative, non-finite and excessive-precision values.
Currency	Exact supported value `CNY`.
Identifiers	Bounded, nonempty IDs with a maximum length of 128 characters.
Additional fields	Reject unrecognized fields, including attempted authorization overrides such as `bypass_mfa` or `admin`.
Approval metadata	Keep `action_id`, digest and tier outside financial tool arguments.

The proposal rationale does not determine risk tier or authorization. Recipient and subscription resolution uses verified backend context rather than identities invented by the model.

Decision fields and stable reason codes

GREEN reads return `READ_ONLY_ACCESS` without confirmation or MFA. Eligible YELLOW transfers return `TRANSFER_WITHIN_DAILY_LIMIT`; subscription cancellations return `SUBSCRIPTION_CANCEL_CONFIRMATION_REQUIRED`. Eligible RED transfers return `TRANSFER_EXCEEDS_DAILY_LIMIT` with both confirmation and MFA required. Policy rejection returns `allowed_to_proceed=False`, independently of tier display.

Validation explicitly rejects negative and zero amounts (including negative zero), sub-cent values such as `100.999`, values outside the configured digit bound, and non-finite inputs such as `NaN` or `Inf`. Unsupported currency strings, including lowercase or whitespace-padded alternatives, are rejected. Empty, null or overlength identifiers are rejected. Extra fields such as `mfa_verified`, `force_immediate`, `user_id`, `action_digest` or `tier` cannot be inserted into canonical tool arguments to alter authority.

Implementation references: `contracts/models.py`, `contracts/tool_schemas.py`, `security/policy.py`, `backend/app/services/actions.py`.

Runtime account checks and security adapter

Transfer schemas require `source_account_id`. The policy compares it with the backend-resolved account before action creation. Read schemas require `account_id`; the chat route validates arguments and invokes policy before querying account data. Locked users retain permitted read access under the default policy.

During confirmation and verification, the backend reconstructs the proposal from stored canonical payload, checks the digest, refreshes the current user and account context, and evaluates policy again. A correct OTP does not override a current policy refusal. A lock introduced after proposal creation still blocks execution.

The security adapter delegates to `security.policy.evaluate_action` by default. Its optional constructor flag accepts only a Boolean or `None`. When environment configuration is used, `USE_MOCK_SECURITY` defaults to `false`; only case-insensitive `true` and `false` are accepted. Invalid configuration raises an error rather than silently selecting a permissive adapter. Explicit mocks support isolated development without changing the default authorization path.

Runtime acceptance conditions

Condition	Required behavior
Account ID differs from context	Reject before reading or creating an executable action.
Account becomes locked after proposal	Reject at confirmation or verification.
Stored payload is changed	Digest or canonical validation prevents execution.
Daily total changes after proposal	Re-evaluate current policy and escalate when needed.
Correct OTP with a rejected policy	Do not execute.
Repeated completed confirmation	Return the existing result without another debit.

Implementation references: `backend/app/services/adapters.py`, `actions.py`, `backend/app/api/chat.py`, `approvals.py`, `backend/tests/test_security_bridge.py`, `test_security_integration.py`.

3 Approval, MFA and transaction safety

Proposal-bound approval

The backend stores canonical tool arguments and computes a SHA-256 digest over the tool and sorted payload JSON. Confirmation references the owned approval ticket and stored digest. The digest binds the approved payload; it is not a digital signature or a replacement for authentication.

Before confirmation or verification can execute an action, the backend reconstructs the proposal from stored data and checks its integrity, ownership, safety-lock state, current balance, subscription eligibility and current permission tier. An earlier YELLOW transfer can escalate to RED after another transfer changes the completed daily total. An already RED ticket is not downgraded.

Verification lifecycle and safety locks

RED confirmation issues a random six-digit challenge bound to the action and owner. The challenge lasts up to five minutes and cannot outlive its ten-minute approval ticket. Expired, consumed or unissued challenges do not authorize execution. Successful verification conditionally consumes the challenge within guarded execution.

Three wrong verification codes or tampered confirmation attempts within a fifteen-minute observation window trigger a persistent user safety lock. Transfers and cancellations are blocked, other pending approvals are retired, and read access remains available. A local operator reset requires a review reason and creates an audit event.

Concurrent requests and financial consistency

Scenario	Implemented control
Duplicate confirmation of one ticket	Claim the action under database protection; completed requests replay the stored outcome rather than execute again.
Competing transfers crossing the daily threshold	Serialize protected account updates and re-evaluate the fresh total; require MFA on escalation.
Competing transfers exceeding available funds	Check current funds within protected execution and roll back partial writes on failure.
Confirm and verify arriving together	Use action status and conditional challenge consumption to reconcile execution.
First transfers after midnight	Roll daily counters forward under execution protection using Asia/Shanghai dates.

Database protection is acquired in user, action and account order. PostgreSQL row protection and SQLite writer serialization support the transaction boundary. Balance updates, ledger entries, audit records and terminal action state are committed together. Failed execution rolls back partial financial writes.

Cancellation and dependent tasks

Users can revoke an unexecuted approval, including one awaiting MFA, without changing balance or subscription status. Completed transfers are not reversed by cancelling a draft. Dependent plan steps advance only after a trusted completion result; failed, rejected or revoked actions stop dependent work. Each new financial action requires its own authorization.

Implementation references: `backend/app/services/action_execution.py`, `daily_transfers.py`, `mfa.py`, `safety_lock.py`, `backend/app/api/approvals.py`, `agent/planning.py`.

Concurrency regression scenarios

The supplied assessment records successful file-backed SQLite checks for the following scenarios. These describe recorded behavior, not newly executed tests in this document revision. PostgreSQL variants require their own configured test environment.

Scenario	Mechanism and recorded result
Same YELLOW ticket confirmed concurrently	Protected action claiming and terminal-state checks limit execution to one transfer ledger entry and one execution audit. A competing request receives the existing outcome or a conflict response.
Two CNY 600 transfers competing for the daily threshold	After one completes, fresh policy sees CNY 600.00 already transferred. The next CNY 600 request exceeds CNY 1,000.00, escalates to `PENDING_MFA` and does not move funds before verification.
Competing transfers exceed available funds	Protected account access and execution-time funds checks allow only affordable transfers. A rejected execution rolls back partial writes and records failure without an additional debit.
Competing RED verifications	Conditional challenge consumption and protected account updates prevent competing verified actions from overspending available funds.
First concurrent requests of a new Shanghai banking day	Protected rollover resets the daily total once, records `DAILY_TRANSFER_TOTAL_RESET`, and counts the new day's completed transfers.

Transaction and replay boundary

The backend execution guard coordinates fresh authorization, balance changes, ledger entries, audits and action state in a single controlled transaction. PostgreSQL uses row protection; SQLite uses its supported writer serialization. The mechanisms are database-specific and should not be treated as evidence that SQLite exercises PostgreSQL row-lock behavior.

The exactly-once financial effect applies to a particular server-issued action and its lifecycle. Independently created proposals are distinct actions and remain subject to fresh daily-total, funds and approval checks. Recovery of a completion reply does not authorize another transfer.

Regression references: `test_same_yellow_action_cannot_execute_twice`, `test_competing_yellow_actions_cannot_bypass_cumulative_mfa`, `test_competing_yellow_actions_cannot_overspend_account`, `test_competing_red_verifications_cannot_overspend_account`, `test_first_concurrent_transfers_of_new_day_reset_once` in `backend/tests/test_security_concurrency.py`.

4 Model boundaries, access protection and audit

Model and tool protections

Threat	Implemented response
Hallucinated balance or completion	Correlate tool results with stored proposal, canonical arguments, account and requested period before reporting facts.
Prompt injection in rationale, merchant or note text	Treat external text as data; enforce schemas and tool whitelist; do not use rationale as authorization.
Model outage or malformed output	Use bounded requests and categorized safe replies; do not create a new proposal on provider failure.
Unauthorized account selection	Resolve account scope in backend context; hosted access pins all visitors to the designated synthetic identity.
Model-generated executable code	Accept structured proposals only. The application does not execute model-generated scripts, shell commands or arbitrary Python.

Typed tool whitelisting is the chosen execution boundary. It is not an operating-system sandbox for arbitrary generated code; such code is outside the application execution path.

Shared deployment access

The Render deployment serves the frontend and backend over HTTPS. A shared HTTP Basic gateway protects the website and API, except the minimal health endpoint. It ignores client-selected demo identity headers, pins requests to `usr_demo_01`, checks browser origins and applies process-local request and write limits.

The Groq API key, database connection string and demo password are private Render environment settings. They are not embedded in frontend code or included in this document. Neon persists application data independently of the web-service filesystem. Reviewers receive only the demo URL and separately supplied shared access credentials.

Audit event coverage

Event group	Recorded event types
Financial execution	`TRANSFER_EXECUTED_ATOMIC`, `SUBSCRIPTION_CANCELLED`
Policy decisions	`ACTION_POLICY_REJECTED`, `ESCALATED_TO_RED`
Verification	`MFA_CHALLENGE_ISSUED`, `VERIFICATION_FAILURE`
Approval lifecycle	`APPROVAL_CANCELLED_BY_USER`, `ACTION_STATUS_OBSERVED`
Safety locks	`SAFETY_LOCK_TRIGGERED`, `SAFETY_LOCK_RESET`, `ACTION_REJECTED_SAFETY_LOCK`
Banking day rollover	`DAILY_TRANSFER_TOTAL_RESET`

Provider failure logs use an event ID, stage, safe category and optional HTTP status. Financial results are calculated deterministically rather than by the language model. Audit records describe backend action and security events; the event set is distinct from a complete transcript of model reasoning or every read authorization.

The application follows an append-oriented audit pattern, but ordinary database records do not provide database-enforced or cryptographic immutability. Initial action proposals are persisted as approval records without a corresponding audit event, and read authorizations are not comprehensively represented in the audit log. Plan pause, stop and operator handoff are not a complete audited operational workflow. These distinctions define the current evidence coverage.

Implementation references: `infra/shared_app.py`, `agent/service.py`, `agent/request_guard.py`, `backend/app/services/audit.py`, `backend/app/api/continuation.py`.

5 Verification evidence and operating conditions

Recorded security checks

The supplied engineering assessment records the following results. They are retained as evidence of that test environment, not presented as a fresh test run or current repository-wide totals. Suite rows overlap and must not be added together as unique coverage.

Suite	Recorded outcome	Scope
`security/tests/test_policy.py`	84 passed	Policy rules, exact thresholds, fixtures and adversarial inputs.
`backend/tests/test_security_bridge.py`	27 passed	Default real adapter, strict configuration and HTTP bridge.
`backend/tests/test_security_integration.py`	16 passed	Execution-time policy, escalation, payload integrity and locks.
`backend/tests/test_security_concurrency.py`	6 passed; 6 PostgreSQL variants not executed	Duplicate execution, threshold, overspending and day rollover on file-backed SQLite.
`backend/tests/test_mfa.py`	18 passed	Challenge lifecycle, verification limits and same-action execution protection.
`tests/test_contracts.py`	14 passed	Shared model and fixture validation.
Combined `backend/tests`	238 passed; 22 PostgreSQL-dependent cases not executed	Broader backend evidence in the supplied assessment; includes overlapping suites above.

The supplied evidence establishes the recorded SQLite checks. PostgreSQL-specific concurrency and load behavior require a separately configured disposable PostgreSQL test environment; deployment on Neon by itself does not establish those test results.

Reproducing the security suites

Use the project's installed Python environment from the repository root. Configure only a disposable database for tests requiring database administration; never point destructive migration or race tests at the hosted demo database.

```
$env:PYTHONPATH = 'backend;.;contracts'
python -m pytest security/tests/test_policy.py -v
python -m pytest backend/tests/test_security_bridge.py -v
python -m pytest backend/tests/test_security_integration.py -v
python -m pytest backend/tests/test_security_concurrency.py -v
python -m pytest backend/tests/test_mfa.py -v
python -m pytest tests/test_contracts.py -v
```

PostgreSQL-specific variants use `MIGRATION_TEST_POSTGRES_ADMIN_URL` when configured. Recorded passing SQLite checks cover duplicate execution, cumulative-threshold escalation, insufficient-funds protection and daily rollover; they do not establish PostgreSQL concurrency under load.

Assessment conclusion

The implemented design separates language understanding from authorization and execution. Recorded checks support fail-closed policy decisions, schema enforcement, action-bound approval, simulated MFA, safety locks and protected financial updates within the stated demonstration environment. The assessment supports a controlled synthetic competition demonstration and does not certify production banking readiness.

Supporting repository documents

The repository contains security documentation, the technical implementation and shared deployment instructions. Test files listed above provide reproducible checks. Review credentials are supplied privately; no actual passwords, API keys or database credentials appear in this assessment.